Question 5: Integrate security into the CI/CD pipeline (DevSecOps): perform vulnerability/dependency scanning and demonstrate secure pipeline practices.

A DevSecOps pipeline was added to the existing Jenkins server: pip-audit (dependency vulnerabilities), bandit (static code analysis) and trivy (container image scan) gate every change. The honest sequence was: blocked -> blocked -> fixed -> passed.

Pipeline stages and the gate story

Build	Dependency audit	Static analysis	Image scan	Result
#1	flask/pytest vulnerable	not reached	not reached	BLOCKED
#2	still vulnerable (no fix yet)	not reached	not reached	BLOCKED
#3	clean after dep bump	B108 + B104 findings	skipped	BLOCKED
#4	clean	clean (B104 skipped)	reported	PASSED

Fix applied after build 3

```
$ git diff app/requirements.txt
-flask==3.1.0
+flask==3.1.3
-pytest==8.3.4
+pytest==9.0.3

# B104 skipped with justification: the app runs inside a container and
# must bind 0.0.0.0 to be reachable from outside the container network
$ bandit -r app/ -q --skip B104

$ pip-audit -r app/requirements.txt
No known vulnerabilities found
```

```
Build #4 console:
No known vulnerabilities found
DevSecOps pipeline: no blocking vulnerabilities found
```

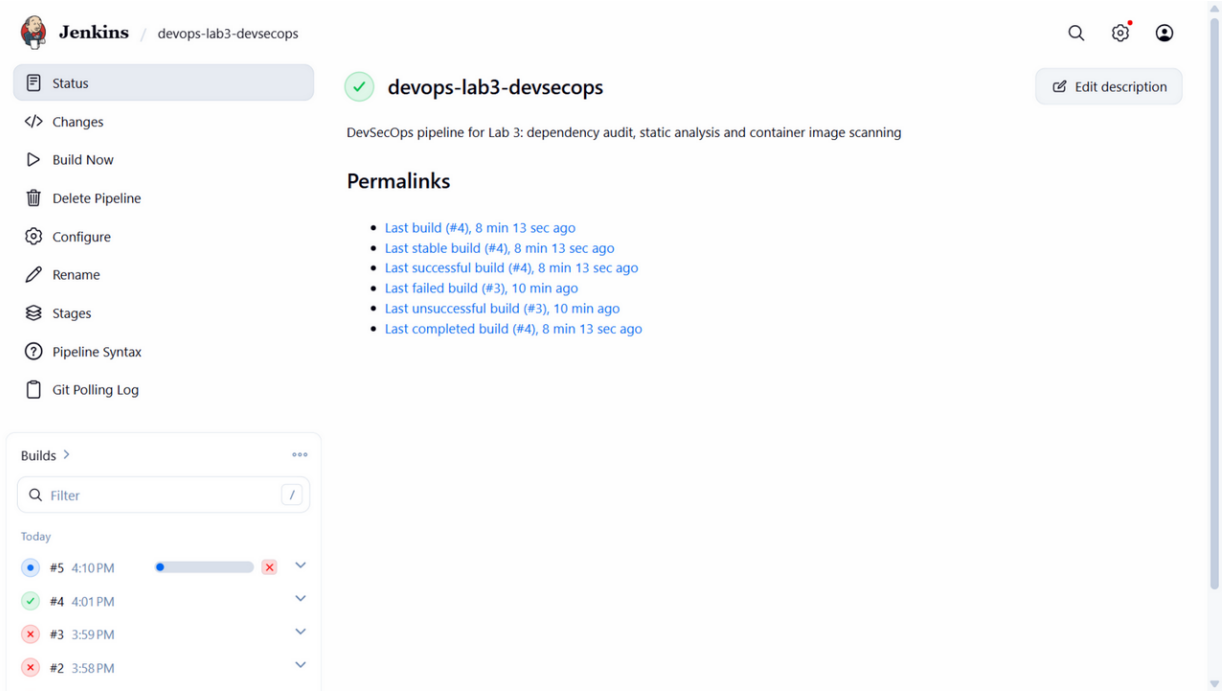


Fig: DevSecOps job: builds 2-3 blocked, build 4 green

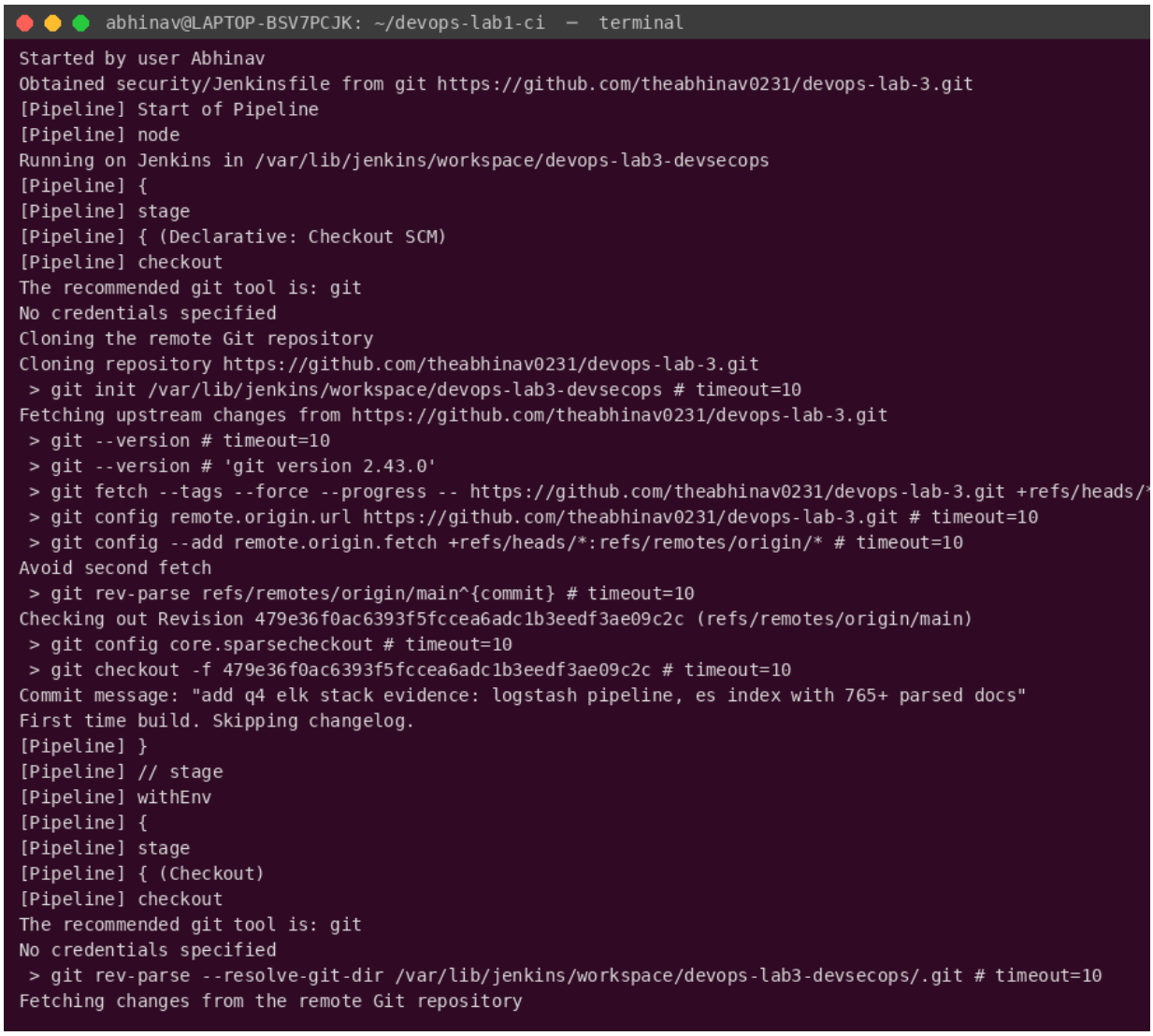


Fig: Build 1 console: pip-audit findings block the pipeline

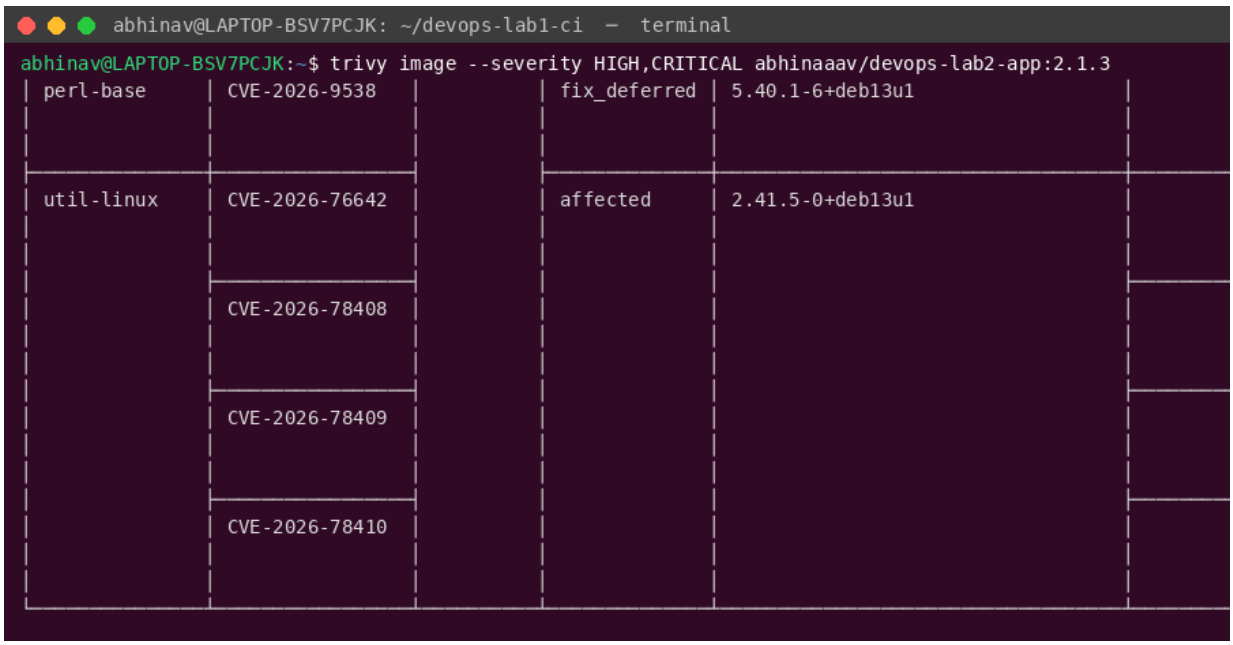


Fig: Vulnerability scan evidence

Secure pipeline practices demonstrated

- Secrets stored as Jenkins credentials (dockerhub token), never committed to the repository.
- Registry tokens passed via --password-stdin instead of CLI arguments.
- Application container runs as a non-root user (appuser).
- Scanners fail the build (exit codes respected) instead of just warning.
- Bandit exceptions are explicit and documented in the Jenkinsfile (B104 justification).

Observations

- pip-audit queries the live PyPI advisory database, so findings reflect current knowledge - flask 3.1.0 had known advisories fixed in 3.1.3.
- Trivy reported real base-image CVEs (util-linux, perl-base) in the lab2 image - recorded as report-only evidence; a hard-fail on OS packages is typically tuned per organization.
- The two failed builds are kept as evidence: the gate provably blocks vulnerable code.

Conclusion

All experiments were performed on Ubuntu 24.04 LTS with Ansible, Terraform, Prometheus, Grafana, the ELK stack and Selenium. The evidence (command logs, configuration files, screenshots and commit history) is committed in the GitHub repository github.com/theabhinav0231/devops-lab-3 under the docs/ directory.